Shared Wallet

Backend Architecture & Security Guide

One combined reference: how to build it, and how to keep it safe

Stack: Python 3.12 · FastAPI · PostgreSQL · Redis · Celery

Approach: mock-first, ledger-based, security by design

Prepared: 16 October 2026

Contents

How to use this guide 4

Part 1 · Foundations 5

1.1 What you are building . . . 5
1.2 Layered security architecture . . . 5
1.3 Development architecture . . . 5
1.4 Technology stack . . . 6
1.5 Project layout . . . 7
1.6 Core modules . . . 7

Part 2 · Identity & access 8

2.1 Authentication — “who is this user?” . . . 8
2.2 User profile & KYC . . . 8
2.3 Groups . . . 8
2.4 Roles and authorization — “what may this user do?” . . . 8

Part 3 · The money engine 10

3.1 Money rules . . . 10
3.2 Immutable double-entry ledger . . . 10
3.3 Atomicity and row locking . . . 10
3.4 Contributions . . . 10
3.5 Merchant payments . . . 11
3.6 Approval system . . . 11

Part 4 · Providers & reliability 12

4.1 Payment-provider abstraction . . . 12
4.2 Webhooks . . . 12
4.3 Idempotency . . . 12
4.4 Reconciliation . . . 13

Part 5 · Product features & platform 14

5.1 Settlements . . . 14
5.2 Notifications . . . 14
5.3 Admin module . . . 14
5.4 File and receipt uploads . . . 14
5.5 Database practices . . . 14
5.6 API standards . . . 14
5.7 Background workers . . . 14

Part 6 · Security hardening 15

6.1 Baseline controls for every endpoint . . . 15
6.2 Common attacks and defences . . . 15
6.3 Data stores . . . 15
6.4 Secret management . . . 16
6.5 Logging and audit . . . 16
6.6 Monitoring and alerting . . . 16
6.7 Fraud prevention . . . 16
6.8 Admin security . . . 16
6.9 Network, CI/CD and dependencies . . . 16
6.10 Incident response . . . 17

Part 7 · Quality & roadmap **18**

7.1 Testing strategy . . . 18
7.2 Implementation roadmap . . . 18
7.3 First MVP scope . . . 18
7.4 Pre-launch security checklist . . . 19
7.5 Easy-to-forget requirements . . . 19

Appendix · Security files to add **20**

How to use this guide

This document merges two plans into one: the **backend build plan** (what to build and in which order) and the **security plan** (how to protect it). Where both documents covered the same topic — authentication, the ledger, idempotency, webhooks, reconciliation, audit logs — the material is now combined in a single place so you read each topic once.

Part	What it covers	Read when
1 · Foundations	Goals, regulatory warning, layered architecture, tech stack, project layout, core modules	Starting the project
2 · Identity & access	Login/OTP/JWT, KYC, groups, roles, authorization rules	Building users and groups
3 · Money engine	Ledger, contributions, payments, approvals, state machines	Building the core of the product
4 · Providers & reliability	Provider adapter, webhooks, idempotency, reconciliation	Connecting payment partners
5 · Product features	Settlements, notifications, admin, files, API standards, workers	Filling out the product
6 · Security hardening	OWASP threats, data stores, secrets, logging, monitoring, fraud, infra, CI/CD, incidents	Before staging and launch
7 · Quality & roadmap	Testing strategy, 10 build phases, MVP, launch checklist	Planning and release

Golden rule of the whole project
Build and prove the **mock transaction flow** first — correct ledger, idempotency, approvals, webhooks and audit logs. Only then plug a real payment sandbox in behind the provider adapter, without changing the core business logic.

Part 1 · Foundations

1.1 What you are building

Shared Wallet should be built as a **modular fintech platform**, not a simple CRUD application. The important pillars are:

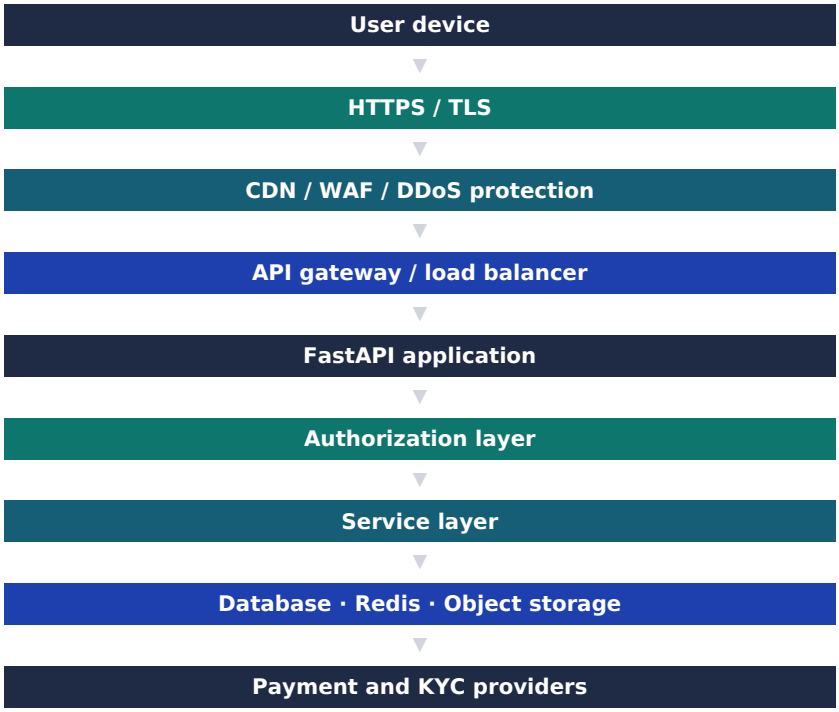
- Authentication, groups and permissions
- Wallet accounting on an immutable ledger
- Contributions and payment approvals
- Partner integrations, webhooks and reconciliation
- Security, audit logs and operational monitoring

> **Regulatory warning (India)**
> Because the app may eventually handle money, do **not** connect directly to banks or UPI providers until you understand the applicable regulatory model and work with an authorized partner. Payment-aggregator activity can involve RBI authorization, security controls, merchant due diligence, escrow requirements, audits, and restrictions on storing sensitive payment credentials.

Security is not an afterthought. It must protect against hackers **and also** against fraud, insider misuse, stolen sessions, fake webhooks, duplicate payments, data leakage and operational mistakes.

1.2 Layered security architecture

Every layer has its own controls. Never rely only on login authentication.



1.3 Development architecture

```
Frontend  ->  FastAPI API  ->  PostgreSQL
              FastAPI API  ->  Redis  ->  Worker
              FastAPI API  ->  Payment Sandbox
              Payment Sandbox  ->  Webhook endpoint (FastAPI)
```

Local services in Docker Compose: **api, postgres, redis, worker, scheduler**. Production adds a reverse proxy/load balancer, TLS, managed PostgreSQL and Redis, object storage, a secret manager, monitoring, error tracking and database backups.

1.4 Technology stack

Layer	Choice	Why
Language / API	Python 3.12+, FastAPI, Pydantic v2, Uvicorn	Typed validation, OpenAPI docs, dependency injection, OAuth2/JWT integrations
Database	PostgreSQL, SQLAlchemy 2.x, asyncpg, Alembic	Transactions, row locks, constraints, migrations
Cache / queue	Redis + Celery (or Dramatiq)	Rate limits, sessions, background jobs
Auth & crypto	pyjwt, pwdlib[argon2], cryptography, slowapi	JWT, hashing, encryption, rate limiting
Provider calls	httpx, tenacity	Timeouts and safe retries
Testing	pytest, pytest-asyncio, factory-boy, faker, testcontainers	Unit, API and real-Postgres integration tests
Code quality	ruff, black, mypy, pre-commit	Format, lint, types, git hooks
Observability	structlog, sentry-sdk, prometheus-fastapi-instrumentator	Logs, errors, metrics
Packaging	Docker, docker-compose	Reproducible environments

Two stack rules that matter for money
1) FastAPI's security helpers do **not** validate JWT signatures for you — your dependency must verify signature, expiry, issuer, audience and algorithm. 2) Never use FastAPI in-process background tasks for critical financial work; use a separate worker. Never auto-retry non-idempotent financial operations.

Example pyproject.toml

```
[project]
name = "shared-wallet-api"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi", "uvicorn[standard]", "pydantic", "pydantic-settings",
    "sqlalchemy", "asyncpg", "alembic", "pyjwt", "pwdlib[argon2]",
    "cryptography", "httpx", "tenacity", "redis", "celery",
    "python-multipart", "slowapi",
]

[project.optional-dependencies]
dev = ["pytest", "pytest-asyncio", "httpx", "factory-boy", "faker",
       "testcontainers", "ruff", "black", "mypy", "pre-commit"]
```

1.5 Project layout

Folder (under app/)	Purpose	Files
core/	Cross-cutting rules	constants, security, permissions, logging, idempotency, errors (+ rate_limit, encryption, secrets)
db/	Database plumbing	base, session, transaction, seed
models/	SQLAlchemy tables	user, session, kyc, group, group_member, invitation, wallet, ledger_account, ledger_transaction, ledger_entry, contribution, payment, payment_approval, settlement, notification, webhook_event, idempotency_key, audit_log
schemas/	Pydantic request/response models	common, auth, user, kyc, group, wallet, contribution, payment, approval, transaction, settlement, notification, errors
routers/	HTTP endpoints	health, auth, users, kyc, groups, members, wallets, contributions, payments, approvals, transactions, settlements, notifications, webhooks, admin
services/	Business logic	auth, user, kyc, group, member, wallet, ledger, contribution, payment, approval, settlement, notification, webhook, reconciliation, audit (+ fraud_service)
repositories/	Data access	user, group, wallet, ledger, payment, transaction
providers/	Partner adapters	interfaces.py + mock/, sandbox/, production/ (otp, kyc, payment)
workers/	Background jobs	celery_app, payment_tasks, notification_tasks, reconciliation_tasks, cleanup_tasks
utils/	Helpers	money, dates, pagination, masking, qr, validators
middleware/	Security middleware	security_headers, request_id, logging
webhooks/	Webhook safety	signature_verification

Outside **app/**: `migrations/` (Alembic), `tests/` (unit, integration, api, fixtures, security), `scripts/` (seed_demo_data, create_admin, reset_database), `docs/` (api-contracts, architecture, ledger-rules, payment-state-machine, security, provider-integration, incident-response, data-retention, threat-model), plus `.env.example, alembic.ini, Dockerfile, docker-compose.yml, pyproject.toml, Makefile, README.md`.

1.6 Core modules

Module	Responsibilities
main.py	Create the app, add routers, CORS, request-ID middleware, exception handlers, OpenAPI metadata, health/readiness endpoints. **No business logic.**
config.py	Read and validate environment variables; separate development / sandbox / staging / production settings. Never commit real secrets.
dependencies.py	get_db(), get_current_user(), require_admin(), require_group_member(), require_group_admin(), require_approver()
middleware	Request ID, structured logging, CORS, rate limiting, security headers, error correlation

```
class Settings(BaseSettings):
    app_name: str = "Shared Wallet API"
    environment: str = "development"
    database_url: str
    redis_url: str
    jwt_secret: str
    jwt_issuer: str
    jwt_audience: str
    payment_provider: str = "mock"
    payment_base_url: str | None = None
    webhook_secret: str | None = None
```

Part 2 · Identity & access

2.1 Authentication — “who is this user?”

Features to implement

- Phone-number login with OTP challenge, expiry, attempt limit and resend cooldown
- Short-lived access token and rotating refresh token
- Logout and token revocation; device/session tracking
- Account freeze; login audit logs; suspicious-login detection

OTP rules

- Never store OTPs in plain text — store a hash or short-lived challenge
- Expire quickly; limit failed attempts; limit requests per phone number **and** per IP
- Never log OTPs and never return them in API responses
- No predictable demo OTP outside development

JWT validation checklist

Always verify — never just decode: **signature, expiration, issuer, audience, algorithm (explicit), token type, token version, and the user's account status**.

2.2 User profile & KYC

User fields: profile, phone, email, city, profile image, KYC status, account status, created date, last login, active devices.



Other KYC statuses: **NEEDS_ATTENTION, REJECTED, EXPIRED**.

Prototype	Production
Use a mock KYC provider. Do **not** collect real Aadhaar, PAN, bank passwords or UPI PIN. Label results “demo verification”.	Use a regulated KYC provider. Store only the necessary verification references, encrypt sensitive data, define retention and deletion policies.

2.3 Groups

- **Actions:** create/update group, invite members, join by code, link or QR, remove/suspend member, change role, freeze or close group, view activity
- **Group rules:** maximum members, contribution model, spending policy, approval threshold, maximum single payment, daily spending limit, whether the initiator can approve their own request, approval expiry time

2.4 Roles and authorization — “what may this user do?”

Role	Can do
OWNER	Manage group, manage members, change policies, freeze or close the group
ADMIN	Manage members, manage group policies, view group activity
APPROVER	Approve or reject eligible payments
MEMBER	Contribute, request payments if allowed, view permitted records
VIEWER	Read-only access

Every protected action is checked on the backend — hiding a button in the frontend is not authorization. For each request validate: user identity, resource ownership, group membership, group role, action permission, object-level permission and field-level permission.

Top API risk: broken object/function-level authorization
A user who changes `/groups/group_123` to `/groups/group_456` in the URL must not gain access to another group. Never trust IDs from the frontend; test horizontal and vertical privilege escalation.

Prevent mass assignment and data over-exposure

- Use separate **input schemas** that contain only fields the user may change — a user must never be able to submit `role`, `is_verified` or `wallet_balance_paise`.
- Never return OTP hashes, refresh tokens, provider secrets, unnecessary full phone numbers, KYC documents, needless internal IDs, or other members' private data.

Part 3 · The money engine

This is the most important part of the backend. Everything here must be correct before any real provider is connected.

3.1 Money rules

- Store money as integer **paise**: ₹2,000 = `200000` paise (`amount_paise: int`)
- Never use floating-point values for money
- Never write `wallet.balance = wallet.balance + amount` — it ruins auditing, reversals, disputes and reconciliation

3.2 Immutable double-entry ledger

Every financial event creates immutable ledger records: contribution credit, merchant payment debit, refund credit, reversal credit/debit, settlement entry, adjustment entry.

Event	Debit	Credit	Amount
Contribution	External contribution account	Group wallet account	₹5,000 (500000 paise)
Merchant payment	Group wallet account	Merchant settlement account	₹2,000 (200000 paise)

Ledger tables

Table	Columns
ledger_accounts	id, owner_type, owner_id, account_type, currency, status, created_at
ledger_transactions	id, transaction_type, reference_type, reference_id, status, idempotency_key, created_at, posted_at
ledger_entries	id, ledger_transaction_id, ledger_account_id, direction, amount_paise, currency, created_at

- Add database constraints so every posted transaction has **balanced** debit and credit entries.
- Never delete financial records — use reversals or controlled corrections.

3.3 Atomicity and row locking

A payment operation updates these records in **one database transaction**, all-or-nothing:

- Payment status
- Ledger transaction and entries
- Wallet reservation
- Audit log

When spending: lock the wallet/account row, **recheck** the available balance inside the transaction, then reserve or debit atomically. This stops two concurrent payments from spending the same money.

3.4 Contributions

CREATED → PENDING → SUCCESS

From **PENDING** a contribution can also end as FAILED, EXPIRED or REVERSED. Features: pending/success/failure/reversal statuses, provider reference, history, duplicate-request protection, webhook processing, balance update.

> **Never trust the frontend**
> Do not mark a contribution successful because the frontend says so. Final state must come from a verified provider response, a verified webhook, or a clearly labelled mock provider (development only).

3.5 Merchant payments

Required: create payment request; validate group status, member permission and available balance; apply spending limits and approval policy; store merchant details; submit to provider; receive final status; support cancellation, refund and reversal.



Allowed transition	Meaning
CREATED → PENDING_APPROVAL	Request passes validation and enters approval
PENDING_APPROVAL → APPROVED / REJECTED / EXPIRED	Approvers decide, or the approval window lapses
APPROVED → SUBMITTED → PROCESSING	Sent to provider, awaiting final result
PROCESSING → SUCCESS / FAILED	Final provider status (verified)
SUCCESS → REVERSED	Only through a defined reversal operation

Forbidden transitions
SUCCESS → APPROVED, FAILED → SUCCESS, REJECTED → PROCESSING — unless a clearly defined business operation authorizes it. Do not deduct the group wallet before the payment is approved and submitted, per your defined accounting policy.

Keep these four things separate

```
initiated_by_user_id        e.g. Priya Mehta
financial_source_wallet_id  e.g. Goa Trip Shared Wallet
merchant_id                 e.g. Restaurant Demo
approved_by_user_ids        e.g. Rahul Sharma, Aisha Khan
```

3.6 Approval system

- Approval threshold, approver eligibility, self-approval restriction
- Duplicate-approval prevention, approval expiry, rejection reason, history, notifications

Step	approved_approvals	remaining_approvals	payment_status
Payment created (2 required)	0	2	PENDING_APPROVAL
Rahul approves	1	1	PENDING_APPROVAL
Aisha approves	2	0	APPROVED

Enforce with a database constraint: `unique(payment_id, approver_user_id)`.

Part 4 · Providers & reliability

4.1 Payment-provider abstraction

Do not write the payment service against one provider. Define an interface and swap implementations: **Mock → Sandbox → Production**.

```
class PaymentProvider(Protocol):
    async def create_collection(...): ...
    async def create_payout(...): ...
    async def get_payment_status(...): ...
    async def verify_webhook(...): ...
    async def refund_payment(...): ...
```

Sandbox integration

A sandbox typically gives a base URL, client ID and secret, a test merchant, test UPI IDs, payment references, webhook URL configuration and success/failure scenarios. Keep all of it inside `app/providers/` and out of source code:

```
PAYMENT_PROVIDER=mock
PAYMENT_SANDBOX_BASE_URL=
PAYMENT_SANDBOX_CLIENT_ID=
PAYMENT_SANDBOX_CLIENT_SECRET=
PAYMENT_WEBHOOK_SECRET=
```

Unsafe third-party consumption

- Validate provider responses (amount, currency, status, reference) like user input
- Verify signatures, use timeouts and allowlisted domains, log provider failures safely

4.2 Webhooks

Webhooks are provider-to-backend callbacks. Handle each one in this order:

#	Step
1	Validate the signature
2	Validate the timestamp, if supplied
3	Check the event ID has not been processed
4	Match the provider reference to your payment
5	Validate amount and currency
6	Validate the expected state transition
7	Process inside a database transaction
8	Store the raw event securely
9	Return success only after safe persistence
10	Retry safely if processing fails

Replay protection: `unique(provider, provider_event_id)`. A duplicate event must never create another ledger entry.

```
webhook_events: id, provider, provider_event_id, event_type, payload_hash,
                payload_json, processing_status, error_message,
                received_at, processed_at
```

4.3 Idempotency

Idempotency prevents duplicate operations when a user double-clicks, a mobile network retries, the frontend times out, a provider sends duplicate webhooks, or a worker repeats a job.

Require `Idempotency-Key: <unique-client-value>` on every financial write: contributions, payments, approvals, submissions, refunds, reversals, settlements.

```
idempotency_keys: id, key, user_id, request_hash, response_status,
                  response_body, created_at, expires_at
```

If the same key arrives again, return the **original response** instead of creating another transaction.

4.4 Reconciliation

A payment can succeed at the provider but fail to update your database (delayed webhook, server crash, network failure, lost response, duplicate event). Reconciliation jobs:



Reconciliation statuses: **MATCHED, MISMATCHED, PENDING_REVIEW, RESOLVED**.

Part 5 · Product features & platform

5.1 Settlements

Settlements are separate from merchant payments: they calculate who owes whom from contributions and expenses. Features: member net positions (“you owe” / “you are owed”), suggested settlements, request, mark as paid, dispute, history.

> **Be honest in the UI**
> Do not claim a settlement is a real bank transfer unless a real payment provider confirms it.

5.2 Notifications

Events: OTP requested, member joined, contribution received, approval requested, payment approved / rejected / succeeded / failed / reversed, settlement reminder, group frozen, account security event. Store them in a notification table and deliver through background jobs.

5.3 Admin module

- Search users; view groups, wallet status, payments; view failed webhooks and reconciliation mismatches
- Freeze groups and users; review disputes; view audit logs; trigger reconciliation; export reports
- **All admin actions are themselves logged.**

5.4 File and receipt uploads

- Limit file size; allow only approved MIME types; validate file *contents*, not just the extension
- Rename files with generated IDs; never trust the original filename; never execute uploads
- Use private object storage (S3-compatible; local only for development) with short-lived signed URLs
- Scan for malware; strip dangerous metadata where needed

5.5 Database practices

- Foreign keys, unique constraints, Alembic migrations, UTC timestamps, soft deletion where audit history is needed
- Database transactions and row locks for money operations; never delete financial records permanently
- Indexes: users.phone_number, groups.owner_id, group_members.group_id / user_id, payments.group_id / status, contributions.group_id, ledger_transactions.reference_id, webhook_events.provider_event_id, notifications.user_id

5.6 API standards

Version every route (`/api/v1/auth, /users, /groups, /payments, /contributions`) and use consistent envelopes. Include pagination, filtering, sorting, request IDs, idempotency keys, OpenAPI docs and explicit response models.

```
// success
{ "success": true, "data": {}, "meta": { "request_id": "req_123" } }

// error
{ "success": false,
  "error": { "code": "INSUFFICIENT_BALANCE",
             "message": "The group wallet does not have enough available balance." },
  "meta": { "request_id": "req_123" } }
```

5.7 Background workers

Use workers for OTP delivery, email/push notifications, webhook retry, reconciliation, settlement reminders, expired approval and invitation cleanup, report generation and file scanning. Never do long-running work inside an API request.

Part 6 · Security hardening

The plan follows the OWASP API Security Top 10: broken authorization, broken authentication, unrestricted resource consumption, SSRF, misconfiguration, poor API inventory and unsafe third-party API consumption.

6.1 Baseline controls for every endpoint

- HTTPS only; authentication on private routes; authorization on every resource
- Request schema validation and response schema filtering
- Rate limiting, request-size limits, pagination limits, timeouts
- Sanitized error messages, CORS restrictions, security headers, request IDs, audit logging

6.2 Common attacks and defences

Threat	Defences
SQL injection	SQLAlchemy parameterized queries / ORM; **never** build SQL by string concatenation (input validation alone is not enough); least-privilege DB user; security testing
Cross-site scripting	Escape user text, sanitize rich text, never render raw HTML, apply a Content Security Policy, validate receipt metadata and descriptions
CSRF	With cookies: SameSite, CSRF tokens, Origin/Referer checks. With Authorization headers: configure CORS carefully and avoid unsafe browser token storage
Broken access control	Membership check on every group resource, role check on every admin action, never trust frontend IDs, test privilege escalation
Brute force	Rate-limit OTP and login; IP and phone throttling; temporary lockouts; alert on unusual attempts
DDoS / resource exhaustion	CDN + WAF, rate limits, body-size and file-size limits, pagination caps, query timeouts, throttle expensive operations, background jobs, DB connection limits (attackers can burn CPU, bandwidth, SMS quota and provider cost)
SSRF	Do not fetch arbitrary user URLs; domain allowlist; block private IP ranges, localhost and cloud-metadata addresses; disable needless redirects; restrict outbound network
Security misconfiguration	Disable debug in production, hide stack traces, remove test routes and unused methods, restrict CORS, rotate secrets, scan infrastructure config, update dependencies
API inventory problems	Maintain API docs, record all hosts, remove deprecated endpoints, protect staging, do not expose /docs publicly in production, scan for forgotten debug routes
Unsafe third-party APIs	See section 4.1 — verify signatures, validate payloads, timeouts, allowlisted domains

6.3 Data stores

PostgreSQL

- Separate application DB user — no owner, superuser, create-database or production-schema-change rights
- Encryption at rest, TLS connections, private network, firewall rules
- Automated backups, point-in-time recovery, migration review, query monitoring, DB audit logs

Redis (sessions, rate limits, OTP challenges, queues, idempotency state)

- Authentication, private network, TLS where supported, key expiry
- No sensitive values unless necessary; separate databases or instances for queues and cache

6.4 Secret management

Never place secrets in Git, frontend code, Docker image layers, error messages, logs, public documentation or screenshots. Use environment variables locally and a secret manager in staging/production, with key rotation, separate secrets per environment and different credentials for each provider.

Secrets include: JWT secret, database and Redis passwords, payment-provider secret, webhook secret, KYC credentials, SMS provider key, object-storage key.

6.5 Logging and audit

Log	Never log
Request ID, user ID, endpoint, status code, duration, error code, payment reference, webhook event ID, permission failures, admin actions	OTP, JWT, refresh token, payment secrets, UPI PIN, bank password, full identity documents, full card data

Create **immutable audit events** for: login, logout, account freeze, group creation, role changes, payment creation / approval / rejection, contribution success, refund, reversal, settlement and admin actions.

6.6 Monitoring and alerting

Alert on: repeated failed OTPs; unusual-location logins; many accounts on one device; bursts of payment requests; repeated failed payments; rapid group creation; large amount changes; multiple approval attempts; webhook signature failures; **ledger imbalance**; provider mismatch; unexpected admin activity; high error rates; database connection spikes.

Tooling: Sentry (exceptions), Prometheus (metrics), Grafana (dashboards), centralized structured logs, email/messaging alerts.

6.7 Fraud prevention

Security stops technical attacks; fraud prevention stops misuse. Implement transaction velocity limits, group-level and member-level limits, a waiting period for new members making large payments, device fingerprinting (with privacy review), suspicious-IP and impossible-travel detection, payment risk scoring, a manual review queue, group/account freeze, a dispute workflow and refund/reversal controls.

> **Risk-based, not one-signal blocking**
> Do not automatically block users because of a single signal. Use risk-based review with clear reason codes.

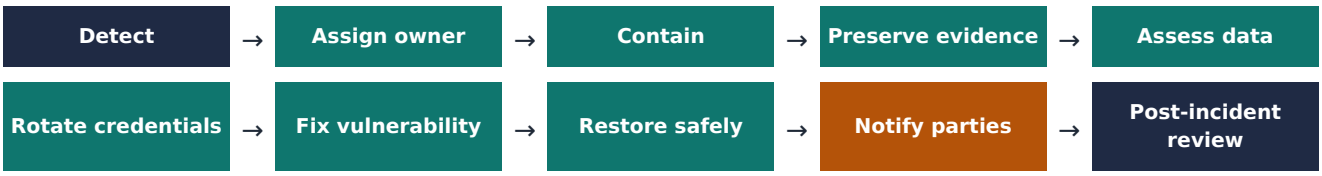
6.8 Admin security

- Separate admin login, mandatory MFA, short sessions, IP allowlisting where possible
- Role separation: read-only support, payment operations, security operations
- Dual approval for sensitive actions; no direct database edits; full admin audit log
- A support admin must not automatically be able to move money

6.9 Network, CI/CD and dependencies

Area	Controls
Network & infrastructure	HTTPS everywhere, TLS certificate management, WAF, DDoS protection, private DB and Redis subnets, firewall rules, restricted security groups, image scanning, non-root containers, minimal OS packages, regular patching, separate dev / staging / sandbox / production networks
CI/CD pipeline	Secret scanning, dependency vulnerability scanning, static analysis, type checking, unit + integration tests, container scanning, migration verification, IaC scanning, manual approval for production. **A failed security check must not deploy.**
Dependencies	Pin versions, use lock files, update and scan for CVEs, remove unused libraries, review transitive dependencies, verify package sources, rebuild images regularly

6.10 Incident response



India-specific note
For India-based entities, review CERT-In reporting obligations and sectoral requirements with legal and security professionals. CERT-In directions state that certain cyber incidents must be reported within **six hours** of noticing them.

Part 7 · Quality & roadmap

7.1 Testing strategy

Level	What to test
Unit	Permission rules, approval logic, balance calculation, settlement calculation, state transitions, idempotency behaviour
Integration	API + PostgreSQL, ledger transaction atomicity, payment-provider adapter, webhook handling, reconciliation
End-to-end	Login → demo KYC → create group → join via invite → contribute → balance updates → create payment → approvers approve → provider returns success → ledger and history update → settlement calculated
Security	test_auth, test_authorization, test_idempotency, test_webhooks, test_rate_limits, test_ledger_integrity

First security milestone
Write tests proving a user **cannot**: access another group, approve unauthorized payments, duplicate a contribution, forge a webhook, or unbalance the ledger. Start with a threat model and a mock environment — not a live payment integration.

7.2 Implementation roadmap

Phase	Build	Done when
1 Foundation	FastAPI app, config, PostgreSQL, Alembic, Docker, health checks, logging, error handling	App boots; migrations run
2 Identity	Mock OTP flow, JWT access + refresh tokens, current user, profile, sessions	Login works end to end
3 Groups	Create/join group, invitations, members, roles, group policies	Roles enforced on backend
4 Ledger	Wallet, ledger accounts and entries, balance calculation, atomicity, reversals	Ledger always balances
5 Contributions	Request, mock provider, state machine, webhook, ledger credit	Contribution credits wallet
6 Payments & approvals	Payment request, permission and balance checks, approvals, state machine, ledger debit	Approved payment debits once
7 Reliability	Idempotency, webhook storage, retry jobs, reconciliation, audit logs	Duplicates and crashes are safe
8 Product features	Notifications, settlements, reports, admin APIs, receipt uploads	Feature-complete on mock
9 Sandbox	Pick partner, get sandbox access, adapter, success/failure + webhook + reconciliation tests, security review	Sandbox flow passes
10 Production readiness	Legal review, partner agreement, security audit, pen-testing, backup/DR, monitoring, incident process, privacy policy, ToS, consent flows	Signed off for live use

7.3 First MVP scope

Do not build everything at once. The MVP is:

- Demo phone login and user profile
- Create/join group, group members, group wallet
- Mock contributions and mock payment requests
- Approval workflow
- Ledger and transaction history
- Basic notifications, mock webhooks, admin logs

First milestone: a complete mock transaction flow with correct ledger behaviour, idempotency, approvals, webhooks and audit logs.

7.4 Pre-launch security checklist

Area	Check
Authentication	☐ OTP is hashed and expires ☐ OTP attempts are limited ☐ Tokens are short-lived ☐ Refresh tokens rotate ☐ Logout revokes sessions ☐ MFA exists for admins
Authorization	☐ Every object access checks ownership/membership ☐ Every admin function checks role ☐ Field-level permissions exist ☐ Unauthorized responses do not leak information
Financial integrity	☐ Money uses integer paise ☐ Ledger entries are immutable ☐ Debits and credits balance ☐ Transactions are atomic ☐ Duplicate operations are blocked ☐ Reversals are supported ☐ Provider reconciliation exists
API protection	☐ HTTPS enforced ☐ Rate limiting active ☐ Request sizes limited ☐ CORS restricted ☐ Debug mode disabled ☐ Errors hide internals ☐ Old APIs removed or protected
Provider integration	☐ Webhook signatures verified ☐ Event IDs deduplicated ☐ Provider responses validated ☐ Amount and currency checked ☐ Timeouts and retries are safe ☐ Sandbox / production credentials separated
Data protection	☐ Secrets managed securely ☐ Sensitive fields encrypted or minimized ☐ Logs redacted ☐ Receipts private and scanned ☐ Backups tested ☐ Data retention documented
Operations	☐ Monitoring active ☐ Alerts configured ☐ Audit logs immutable ☐ Incident response documented ☐ Security testing completed ☐ Dependencies scanned

7.5 Easy-to-forget requirements

Idempotency keys · webhook signature verification and replay protection · reconciliation jobs · immutable ledger · paise handling · time zones · request IDs · audit logs · role-based authorization · account and group freezing · payment expiration · provider timeout handling · reversal and refund handling · duplicate-member prevention · invite expiry · file scanning · data retention · database backups · secret rotation · incident response · API versioning · rate limiting · monitoring · admin access controls · legal and compliance review.

Appendix · Security files to add

Area	Files
Core	app/core/security.py, permissions.py, rate_limit.py, idempotency.py, encryption.py, secrets.py
Middleware	app/middleware/security_headers.py, request_id.py, logging.py
Webhooks & services	app/webhooks/signature_verification.py; app/services/fraud_service.py, reconciliation_service.py, audit_service.py
Security tests	tests/security/test_auth.py, test_authorization.py, test_idempotency.py, test_webhooks.py, test_rate_limits.py, test_ledger_integrity.py
Docs	docs/security.md, incident-response.md, data-retention.md, threat-model.md